

Middleware

LangChain Agent Middleware is a framework feature designed to provide fine-grained control, security, and context engineering inside an agent's execution loop.

Middleware attaches to distinct phases of the agent loop. These can be implemented using class-based structures or simple function decorators:

Node-style Hooks

- `@before_agent` : Runs once when the agent is initially invoked. Ideal for retrieving long-term user profile records or database states
- `@before_model` : Intercepts execution before every LLM call. Perfect for modifying prompts, truncating chat histories, or checking token budgets.
- `@after_model` : Evaluates the model output immediately after it returns. Used for validation, extraction structured data, or formatting results.
- `@after_agent` : Triggers once after the agent finishes its complete cycle. Useful for calculating total costs, tracking metrics, or logging cleanups.

Wrap-Style Hooks

- `@wrap_model_call` : Enclose the underlying model execution block. Essential for structural behaviors like exponential backoff retries, alternative model fallbacks, or prompt caching
- `@wrap_tool_call` : Enclose tool executions. Used to intercept arguments, block hazardous scripts, or evaluate performance.

LangChain also provides production-ready, configurable Prebuilt Middleware addressing common AI bottlenecks:

`SummarizationMiddleware` = Compressing older chat history when approaching context token ceilings. --> Summarize older chat to save space for new chat history

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware

# Single condition: trigger if tokens >= 4000
agent = create_agent(
    model="gpt-5.4",
    tools=[your_weather_tool, your_calculator_tool],
    middleware=[
        SummarizationMiddleware(
            model="gpt-5.4-mini",
            trigger=("tokens", 4000),
```

```

        keep=("messages", 20),
    ),
],
)

# Multiple conditions: trigger if number of tokens >= 3000 OR messages >= 6
agent2 = create_agent(
    model="gpt-5.4",
    tools=[your_weather_tool, your_calculator_tool],
    middleware=[
        SummarizationMiddleware(
            model="gpt-5.4-mini",
            trigger=[
                ("tokens", 3000),
                ("messages", 6),
            ],
            keep=("messages", 20),
        ),
    ],
)

# AND logic: trigger only when tokens >= 4000 AND messages >= 10
agent3 = create_agent(
    model="gpt-5.4",
    tools=[your_weather_tool, your_calculator_tool],
    middleware=[
        SummarizationMiddleware(
            model="gpt-5.4-mini",
            trigger={"tokens": 4000, "messages": 10},
            keep=("messages", 20),
        ),
    ],
)

# Combine AND and OR: trigger if (tokens >= 5000 AND messages >= 3)
# OR (tokens >= 3000 AND messages >= 6)
agent4 = create_agent(
    model="gpt-5.4",
    tools=[your_weather_tool, your_calculator_tool],
    middleware=[
        SummarizationMiddleware(
            model="gpt-5.4-mini",
            trigger=[
                {"tokens": 5000, "messages": 3},
                {"tokens": 3000, "messages": 6},
            ],
            keep=("messages", 20),
        ),
    ],
)

```

```
# Using fractional limits
agent5 = create_agent(
    model="gpt-5.4",
    tools=[your_weather_tool, your_calculator_tool],
    middleware=[
        SummarizationMiddleware(
            model="gpt-5.4-mini",
            trigger=("fraction", 0.8),
            keep=("fraction", 0.3),
        ),
    ],
)
```

HumanInTheLoopMiddleware = Pauses execution and requests human approval/edits before dangerous tools step (e.g., sending email)

PIIMiddleware = Scans and redacts sensitive data (emails, credit cards, IPs) from outputs or inputs

ModeCallLimitMiddleware / ToolCallLimitMiddleware = Hard-caps call counts per run or conversation thread to stop infinite loops and budget overruns.

ModelFallbackMiddleware = Swaps to an alternative LLM backend automatically if the primary provider hits a rate limit or fails

TodoListMiddleware = Empowers the agents with internal task planning by injecting custom step-tracking tools into the prompt.

More on: <https://docs.langchain.com/oss/python/langchain/middleware/built-in>

<u>Summarization</u>	Automatically summarize conversation history when approaching token limits.
<u>Human-in-the-loop</u>	Pause execution for human approval of tool calls.
<u>Model call limit</u>	Limit the number of model calls to prevent excessive costs.
<u>Tool call limit</u>	Control tool execution by limiting call counts.
<u>Model fallback</u>	Automatically fallback to alternative models when primary fails.
<u>PII detection</u>	Detect and handle Personally Identifiable Information (PII).
<u>To-do list</u>	Equip agents with task planning and tracking capabilities.
<u>LLM tool selector</u>	Use an LLM to select relevant tools before calling main model.
<u>Tool retry</u>	Automatically retry failed tool calls with exponential backoff.
<u>Model retry</u>	Automatically retry failed model calls with exponential backoff.
<u>LLM tool emulator</u>	Emulate tool execution using an LLM for testing purposes.
<u>Context editing</u>	Manage conversation context by trimming or clearing tool uses.
<u>Shell tool</u>	Expose a persistent shell session to agents for command execution.
<u>File search</u>	Provide Glob and Grep search tools over filesystem files.
<u>Filesystem</u>	Provide agents with a filesystem for storing context and long-term memories.
<u>Subagent</u>	Add the ability to spawn subagents.